Templates allow ACME's identity team to enforce consistent Cedar policies while binding a few variables at instantiation time—keeping logic centralized and minimizing drift. A service or admin workflow supplies the parameters, resulting in a standard, type-checked policy within the set.

```
// Template: document-view-template
permit(
  principal in ?readers_team,
  action == ACME::"doc:view",
  resource in ACME::Document
 )
when { resource.owner == ?owner };
```

An example template instantiation input could look like this:

```
{
  "template": "document-view-template",
  "params": {
    "readers_team": "ACME::Team::\"team:eng-doc-readers\"",
    "owner": "ACME::Employee::\"emp:alice\""
  }
}
```

Use templates when you need multiple nearly identical rules that only differ by identifiers or specific, narrow conditions, such as per-workspace reader teams, per-customer collections, or per-environment scopes. Choose instance policies (hand-written) when a case genuinely differs in structure, not just in values.

Here are some useful operational tips:

- **Validate parameters**—ensure they exist, are the correct type, and match tenancy before instantiation to prevent broken policies.
- **Name and version**—provide templates with semantic versions and document which instance came from which template version for audit purposes.
- **Review changes**—treat template modifications as code; run test vectors and use staged rollouts to prevent widespread regressions.